



Pontifícia Universidade Católica do Rio Grande do Sul
Confiabilidade e Segurança de Software
98G08-04

Lista de Exercícios
Cifras de Fluxo, OTP, LFSR e ChaCha
(Com Respostas)

Prof. Iaçanã Ianiski Weber

Observação: esta versão contém os enunciados traduzidos e uma proposta de solução. Em questões discursivas, respostas equivalentes também podem ser consideradas corretas.

Exercício 1

A cifra de fluxo descrita na Definição 2.1.1 pode ser facilmente generalizada para operar em alfabetos diferentes do binário. Para cifragem manual, uma versão especialmente útil é uma cifra de fluxo que opera sobre letras.

1. Desenvolva um esquema que opere com as letras A, B, ..., Z, representadas pelos números 0, 1, ..., 25. Como é a chave (fluxo de chave)? Quais são as funções de cifragem e decifragem?
2. Decifre o seguinte texto cifrado:
bsaspp kkuosr
que foi cifrado usando a chave:
rsidpy dkawoa
3. Como o jovem foi assassinado?

Associe $A \mapsto 0, B \mapsto 1, \dots, Z \mapsto 25$.

Sejam:

$m_i \in \{0, \dots, 25\}$ o símbolo do texto em claro,
 $k_i \in \{0, \dots, 25\}$ o símbolo da chave/fluxo de chave,
 $c_i \in \{0, \dots, 25\}$ o símbolo do texto cifrado.

A chave é uma sequência de letras (ou números em $\{0, \dots, 25\}$), por exemplo:

$$k = (k_0, k_1, k_2, \dots).$$

A função de cifragem é

$$c_i = (m_i + k_i) \bmod 26.$$

A função de decifragem é

$$m_i = (c_i - k_i) \bmod 26.$$

Aplicando a decifragem a `bsaspp kkuosr` com a chave `rsidpy dkawoa`, obtemos:

`kaspar hauser.`

Portanto, a resposta do item 3 é: o jovem era Kaspar Hauser, e ele foi morto por apunhalamento (esfaqueado).

Exercício 2

Suponha que armazenemos uma chave de uso único (*one-time key*) em um DVD com capacidade de 1 Gbyte. Discuta as implicações *reais* de um sistema de *one-time pad* (OTP). Aborde questões como o ciclo de vida da chave, o armazenamento da chave durante o ciclo de vida e após o fim desse ciclo, a distribuição da chave, a geração da chave, etc.

Embora o OTP ofereça sigilo perfeito em teoria, seu uso prático traz dificuldades severas:

- **Geração da chave:** a chave precisa ser verdadeiramente aleatória, não apenas pseudoaleatória. Gerar 1 Gbyte de aleatoriedade de alta qualidade já é um desafio operacional.
- **Distribuição da chave:** remetente e destinatário precisam receber exatamente a mesma chave por um canal seguro *antes* da comunicação. Isso desloca o problema da segurança para a distribuição física ou para um canal previamente seguro.
- **Tamanho da chave:** a chave precisa ter pelo menos o mesmo tamanho da mensagem total a ser protegida. Para volumes grandes, isso implica armazenamento e logística muito pesados.
- **Controle de uso:** cada trecho da chave só pode ser usado uma única vez. É preciso manter contadores, offsets ou marcações para garantir que nunca haja reutilização.
- **Sincronização:** emissor e receptor precisam saber exatamente qual parte da chave foi consumida. Um desalinhamento faz a decifragem falhar.
- **Armazenamento durante a vida útil:** enquanto a chave ainda não foi usada, ela precisa ser armazenada com proteção física e lógica comparável à da própria informação secreta.

- **Descarte após uso:** após o uso, a chave precisa ser destruída de forma confiável. Se cópias permanecerem em disco, backup, cache, RAM, mídia óptica ou imagens forenses, o sigilo pode ser perdido.
- **Escalabilidade:** para e-mails, mensageria instantânea e comunicação massiva, o custo operacional de gerar, transportar, armazenar, sincronizar e destruir chaves é impraticável.

Em resumo, o OTP é extremamente seguro do ponto de vista teórico, mas impõe um custo logístico e operacional tão alto que, na prática, quase sempre é substituído por cifras simétricas modernas com chaves curtas e geradores pseudoaleatórios robustos.

Exercício 3

Suponha uma cifra de um único bit (OTP) com uma chave curta de 128 bits. Essa chave é então usada periodicamente para cifrar grandes volumes de dados. Descreva como funciona um ataque que quebra esse esquema.

Se a chave de 128 bits é repetida periodicamente, então o esquema deixa de ser um OTP e passa a ser uma cifra de chave repetida, análoga a uma cifra de Vigenère sobre o alfabeto binário.

Se $k_0, \dots, k_{127}$ é a chave, então:

$$c_i = m_i \oplus k_{i \bmod 128}.$$

O ataque explora o fato de que os mesmos 128 bits de chave são reutilizados indefinidamente:

- Bits em posições separadas por múltiplos de 128 são cifrados com o mesmo bit de chave.
- Ao agrupar os dados por posição módulo 128, o atacante obtém muitas amostras cifradas com o mesmo bit de chave.
- Como textos reais têm redundância estatística, essas amostras podem ser analisadas para inferir os bits da chave. Em texto ASCII/inglês, isso é ainda mais fácil.
- Se o atacante conhece algum bloco de 128 bits do texto em claro e o bloco cifrado correspondente, então recupera imediatamente toda a chave:

$$k_i = m_i \oplus c_i, \quad i = 0, \dots, 127.$$

- Uma vez conhecida a chave de 128 bits, todo o restante do tráfego pode ser decifrado:

$$m_i = c_i \oplus k_{i \bmod 128}.$$

Logo, a reutilização periódica da chave destrói completamente a segurança do esquema.

Exercício 4

À primeira vista, parece que uma busca exaustiva de chave é possível contra um sistema OTP. É dada uma mensagem curta, digamos 5 caracteres ASCII representados por 40 bits, que foi cifrada usando um OTP de 40 bits. Explique *exatamente* por que uma busca exaustiva não terá sucesso, mesmo que recursos computacionais suficientes estejam disponíveis. Isso parece um paradoxo, já que sabemos que o OTP é incondicionalmente seguro. Isto é, explique por que um ataque de força bruta não funciona.

No OTP, para um texto cifrado c e qualquer texto em claro candidato m de 40 bits, existe *exatamente uma* chave de 40 bits que transforma m em c :

$$k = c \oplus m.$$

Portanto, ao fazer força bruta sobre todas as chaves possíveis, o atacante não obtém “a” mensagem correta. Ele obtém *todas* as mensagens possíveis, cada uma associada a uma chave diferente.

Ou seja:

- cada chave candidata produz um texto em claro candidato;
- para qualquer texto em claro de 40 bits, há uma chave que o explica perfeitamente;
- o texto cifrado, sozinho, não fornece informação para distinguir qual desses textos em claro é o verdadeiro.

Assim, a busca exaustiva *enumera hipóteses*, mas não oferece um critério interno para selecionar a correta. O problema não é falta de computação; é falta de informação.

Não há paradoxo: a força bruta funciona como procedimento de enumeração, mas falha como procedimento de *decisão*. O OTP é incondicionalmente seguro justamente porque o texto cifrado não revela informação suficiente para preferir uma chave em relação às demais.

Exercício 5

O OTP pode ser usado para cifrar dados de comprimento arbitrário ao cifrar símbolos binários $x_i \in \{0, 1\}$. Decifre manualmente o seguinte texto cifrado. O texto cifrado é dado em notação hexadecimal:

26 34 05 18 0c 06 07 15 1c 2a 13 3c 0c 23 04 27 07 27 18

A chave é dada por:

6a 51 71 6b 49 68 64 67 65 5a 67 68 64 4a 77 65 68 48 73

Calcule o texto em claro, que está codificado em símbolos ASCII.

Como o OTP binário usa XOR:

$$m_i = c_i \oplus k_i.$$

Fazendo o XOR byte a byte, obtemos:

LetsEncryptThisBook

Portanto, o texto em claro é LetsEncryptThisBook.

Exercício 6

O OTP oferece segurança demonstrável. Descreva duas grandes desvantagens do OTP que o tornam impraticável para a maioria das aplicações, como cifragem de e-mails ou mensagens instantâneas.

Duas desvantagens centrais são:

1. **Distribuição e armazenamento da chave:** a chave precisa ser tão grande quanto a quantidade total de dados a proteger e deve ser distribuída previamente por um canal seguro.
2. **Uso único rigoroso:** qualquer reutilização da chave compromete a segurança. Na prática, controlar geração, sincronização, consumo e destruição da chave é muito difícil.

Esses dois pontos tornam o OTP impraticável para sistemas de comunicação usuais.

Exercício 7

Suponha que temos uma cifra de fluxo cujo período é bastante curto. Sabemos que esse período tem entre 150 e 200 bits de comprimento. Assumimos que não sabemos mais nada sobre os detalhes internos da cifra de fluxo. Em particular, não devemos supor que ela seja um LFSR simples. Para simplificar, assuma que texto em inglês, em formato ASCII, está sendo cifrado.

Descreva em detalhe como uma cifra desse tipo pode ser atacada. Especifique exatamente o que Oscar precisa conhecer em termos de texto em claro/texto cifrado, e como ele pode decifrar todo o texto cifrado.

Se o período da sequência de chave é curto, então existe um $T \in [150, 200]$ tal que

$$z_{i+T} = z_i.$$

Logo, a cifra efetivamente reutiliza a mesma sequência de chave repetidamente.

Há duas formas principais de ataque:

(a) Ataque com texto conhecido. Se Oscar conhece um trecho de texto em claro e o texto cifrado correspondente, então ele recupera o trecho de fluxo de chave por

$$z_i = m_i \oplus c_i.$$

Se esse trecho conhecido cobrir pelo menos um período completo (na prática, algo em torno de 200 bits consecutivos já é suficiente), Oscar recupera todo o período da chave. Depois disso, basta repetir a sequência e decifrar tudo:

$$m_i = c_i \oplus z_{i \bmod T}.$$

(b) Ataque somente com texto cifrado. Mesmo sem conhecer o texto em claro, o ataque ainda é viável porque o texto é ASCII em inglês e, portanto, tem muita redundância:

- o bit mais significativo de cada caractere ASCII é 0;
- espaço é muito frequente;
- letras e sinais comuns têm distribuições estatísticas não uniformes.

Procedimento:

1. Testar cada período candidato $T \in \{150, \dots, 200\}$.
2. Para cada T , agrupar os bits do texto cifrado por posição módulo T . Cada grupo foi cifrado pelo mesmo bit de chave.
3. Para cada posição do período, tentar os dois valores possíveis do bit de chave (0 ou 1) e verificar qual escolha produz bits de texto em claro mais compatíveis com ASCII/inglês.

4. Escolher o valor de T e o período de chave que maximizam a plausibilidade linguística do texto resultante.

Uma vez recuperado um período da sequência de chave, todo o restante do texto cifrado pode ser decifrado por repetição periódica da chave.

Em resumo, Oscar precisa de:

- **ou** um trecho conhecido de texto em claro/texto cifrado com comprimento de pelo menos um período;
- **ou** uma quantidade suficientemente grande de texto cifrado em inglês ASCII para explorar periodicidade e redundância estatística.

Exercício 8

É dada uma cifra de fluxo baseada em um único LFSR como gerador de fluxo de chave. O LFSR tem grau 256.

1. Quantos pares de bits de texto em claro/texto cifrado são necessários para lançar um ataque bem-sucedido?
2. Descreva todos os passos do ataque em detalhe e desenvolva as fórmulas que precisam ser resolvidas.
3. Qual é a chave nesse sistema? Por que não faz sentido usar o conteúdo inicial do LFSR como chave ou como parte da chave?

1. São suficientes **512 pares consecutivos** de bits de texto em claro/texto cifrado. A partir deles, o atacante obtém 512 bits consecutivos do fluxo de chave:

$$z_i = m_i \oplus c_i, \quad i = 0, \dots, 511.$$

2. Seja a recorrência linear do LFSR:

$$z_{n+256} = a_0 z_n \oplus a_1 z_{n+1} \oplus \dots \oplus a_{255} z_{n+255},$$

com coeficientes $a_0, \dots, a_{255} \in \{0, 1\}$.

Usando os 512 bits conhecidos do fluxo de chave, montamos 256 equações lineares sobre $\text{GF}(2)$:

$$z_{256+i} = a_0 z_i \oplus a_1 z_{i+1} \oplus \dots \oplus a_{255} z_{i+255}, \quad i = 0, \dots, 255.$$

Essas 256 equações permitem resolver os 256 coeficientes desconhecidos $a_0, \dots, a_{255}$. Com isso, recupera-se o polinômio de realimentação do LFSR.

Depois de recuperar os coeficientes, os primeiros 256 bits

$$z_0, z_1, \dots, z_{255}$$

servem como estado inicial para reproduzir todo o fluxo de chave futuro e, com isso, decifrar qualquer texto:

$$m_i = c_i \oplus z_i.$$

3. A informação secreta relevante nesse sistema é a **lei de realimentação** do LFSR, isto é, seus coeficientes (ou, equivalentemente, o polinômio de conexão).

Não faz sentido usar o conteúdo inicial do LFSR como chave, ou como parte da chave, porque esse conteúdo é rapidamente exposto pela própria saída. Em muitas convenções, os primeiros bits de saída correspondem diretamente ao estado inicial; de forma mais geral, ele pode ser recuperado de um pequeno trecho do fluxo de chave. Assim, o estado inicial é um valor transitório muito fraco como segredo de longo prazo.

Observação: se o polinômio de realimentação já fosse público e apenas o estado inicial fosse secreto, então bastariam 256 bits consecutivos do fluxo de chave para recuperar esse estado.

Exercício 9

Vamos agora observar mais de perto a função de quarto de rodada (*quarter-round*) do ChaCha, isto é, $QR(a, b, c, d)$. Qual é a saída de QR para a seguinte entrada?

$$a = 0x00000001$$

$$b = 0x00000000$$

$$c = 0x00000000$$

$$d = 0x00000000$$

Aplicando a função quarter-round do ChaCha:

$$a = a + b; \quad d = d \oplus a; \quad d \lll 16$$

$$c = c + d; \quad b = b \oplus c; \quad b \lll 12$$

$$a = a + b; \quad d = d \oplus a; \quad d \lll 8$$

$$c = c + d; \quad b = b \oplus c; \quad b \lll 7$$

obtemos a saída final:

$$a = 0x10000001$$

$$b = 0x80808808$$

$$c = 0x01010110$$

$$d = 0x01000110$$

Logo,

$$\begin{aligned} & QR(0x00000001, 0x00000000, \\ & \quad 0x00000000, 0x00000000) \\ &= (0x10000001, 0x80808808, \\ & \quad 0x01010110, 0x01000110). \end{aligned}$$